

MoodSync Studio: Multimodal **Emotion and Sentiment Analysis**

Project Report

Project name: MoodSync Studio / MSyncStudio V1.2

Project type: Deep learning and generative AI application

Primary interface: Streamlit web application

Core task: Multimodal facial emotion, language sentiment, and mismatch detection

Submitted by: Sidharth Arakkan

Abstract

MoodSync Studio is a multimodal AI application that analyses a person's visible facial emotion together with typed or spoken language. The system accepts a face image or a clean webcam capture, combines it with typed text or an audio transcript, and produces an aligned or mismatched verdict. It also provides supporting visualisations such as image attention heatmaps, token attention charts, session history, and a practical action-plan view.

The project uses pretrained models for each major modality: a Vision Transformer facial emotion model for images, a RoBERTa sentiment model for language, and OpenAI Whisper for speech-to-text transcription. Pretrained models were selected after initial attempts to train custom models with different emotion datasets proved impractical on the available hardware. Training modern transformer and vision models requires significant GPU memory, long training time, and large curated datasets. The final design therefore focuses on integrating reliable pretrained components into a clear, explainable multimodal pipeline.

1. Introduction

Human emotion is often communicated through more than one signal. A person may smile while writing or saying something negative, or may use neutral words while their facial expression suggests stress. Single-modal systems can miss this type of contradiction because they analyse only one channel of information. MoodSync Studio addresses this limitation by combining visual and language signals.

The main goal is not to diagnose emotion in a clinical sense. Instead, the project demonstrates how deep-learning models from different modalities can be integrated into one interactive application. The app estimates facial mood, estimates language sentiment, maps both outputs into a shared polarity space, and then decides whether the two signals are aligned or mismatched.

This approach follows the broader direction of multimodal machine learning, where systems process and relate information from multiple modalities. Baltrusaitis, Ahuja, and Morency describe multimodal machine learning as a field concerned with representation, alignment, fusion, translation, and co-learning across modalities [8]. MoodSync Studio focuses mainly on representation, alignment, and fusion.

2. Objectives

The project objectives are:

- Build an interactive AI application for facial emotion and language sentiment analysis.
- Support both image-upload and webcam-capture workflows.
- Support typed text, recorded audio, and uploaded audio.
- Make text mandatory only when no usable audio transcript exists.
- Detects whether facial emotion and language sentiment are aligned or mismatched.
- Provide explainable outputs through heatmaps, token attention, confidence charts, and summaries.

- Store recent analyses during the current session.
 - Generate a practical action plan from the latest analysis.
 - Keep the system runnable on normal local hardware by using pretrained models.
-

3. Scope

MoodSync Studio is a prototype and educational demonstration of multimodal AI. It is designed for local experimentation and project presentation. It is not a medical tool, a lie detector, or a psychological diagnosis system.

The current version analyses:

- One uploaded image or one captured webcam frame.
- Typed text or a transcript generated from recorded/uploaded audio.
- Facial emotion probability distribution.
- Language sentiment probability distribution.
- Alignment or mismatch between visual and language signals.

The system does not currently perform:

- End-to-end model training on a large aligned multimodal dataset.
 - Clinical validation.
 - Long continuous video timeline analysis in the main Streamlit workflow.
 - Audio prosody emotion recognition such as pitch, tone, rhythm, or speech rate.
-

4. System Architecture

```
text

User input
|
|-- Face image or webcam capture
|   -> OpenCV face detection
|   -> ViT facial emotion classifier
|   -> Emotion label + probabilities + attention heatmap
|
|-- Typed text or audio input
|   -> Whisper transcription when audio is used
|   -> RoBERTa sentiment classifier
|   -> Sentiment label + probabilities + token attention

Visual polarity + Language polarity
-> Fusion layer
-> ALIGNED or MISMATCH
-> Summary, charts, history, action plan
```

4.1 Main Components

Component	File	Responsibility
User interface	app.py	Streamlit tabs, UI layout, input handling, charts, history, reports
Image emotion	image_emotion.py	Face detection, ViT inference, annotated image, attention heatmap
Text sentiment	text_sentiment.py	RoBERTa sentiment inference, clause splitting, token attention

Audio transcription	audio_transcribe.py	Whisper transcription from recorded or uploaded audio
Fusion	fusion.py	Rule-based mismatch logic and optional learned fusion model
Summary generation	generator.py	Human-readable natural-language explanation
Model loading	utils.py	Cached pretrained model loading and label maps
Webcam processor	webcam_live.py	Alternate webcam processor retained for compatibility

5. Methodology

5.1 Facial Emotion Recognition

The image pipeline starts with a face image from either image upload or webcam capture. OpenCV is used to detect face regions using Haar cascade face detection. OpenCV's documentation describes the Cascade Classifier as a method that loads pretrained cascade files and uses `detectMultiScale` to return boundary rectangles for detected faces [7].

After image preparation, the project uses the pretrained Hugging Face model `dima806/facial_emotions_image_detection`. This model is tagged as an image-classification, Transformers, PyTorch, and ViT model, and its model card identifies `google/vit-base-patch16-224-in21k` as its base model [2]. The model predicts seven emotion classes:

- Angry
- Disgust
- Fear

- Happy
- Sad
- Surprise
- neutral

The underlying Vision Transformer concept is based on splitting an image into patches and applying a transformer architecture to those patch embeddings. Dosovitskiy et al. showed that a pure transformer applied directly to image patch sequences can perform well on image classification tasks when pretrained on large datasets and transferred to downstream tasks [1].

5.2 Text Sentiment Analysis

The language pipeline accepts typed text directly or uses an audio transcript generated by Whisper. The project uses cardiffnlp/twitter-roberta-base-sentiment-latest, a pretrained RoBERTa sentiment model. The model card states that it is a RoBERTa-base model trained on approximately 124 million tweets and fine-tuned for sentiment analysis with the TweetEval benchmark [4]. Its labels are:

- Negative
- Neutral
- positive

RoBERTa was selected because it is a strong transformer encoder architecture for language classification. Liu et al. describe RoBERTa as a robustly optimized BERT pretraining approach and note that language model pretraining is computationally expensive and sensitive to hyperparameters and data size [3]. This directly supports the project's decision to use pretrained NLP models instead of training from scratch.

The implementation also splits text into clauses around punctuation and contrast words such as "but", "however", and "although". This helps mixed-emotion input by allowing the sentiment model to evaluate separate clauses and then average class scores.

5.3 Audio Transcription

For audio input, users can record audio in the browser or upload an audio file. Streamlit's `st.audio_input` provides a microphone widget and defaults to a 16000 Hz sample rate, which the documentation identifies as suitable for speech recognition [6]. The app also allows uploaded .wav, .mp3, and .m4a files.

The audio module uses OpenAI Whisper. Whisper is a speech recognition system trained with large-scale weak supervision. The official repository lists multiple model sizes; the base model has 74 million parameters and approximately 1 GB VRAM requirement [5]. MoodSync Studio uses Whisper base because it is a practical balance between capability and local resource usage.

The audio transcript is passed into the same RoBERTa sentiment pipeline used for typed text. This keeps the fusion design simple because both text and audio are converted into the same language sentiment representation.

5.4 Fusion and Mismatch Detection

The default fusion layer maps facial emotion into sentiment polarity:

Emotion	Mapped sentiment
happy	positive
surprise	positive
neutral	neutral
angry	negative
disgust	negative
fear	negative
sad	negative

The mapped visual sentiment is compared with the language sentiment:

```
python

mapped_vis = emotion_to_sentiment.get(vis_emotion, "neutral")
mismatch = mapped_vis != txt_sentiment
result = "MISMATCH" if mismatch else "ALIGNED"
```

This rule-based approach is intentionally explainable. Users can understand why a mismatch was produced: the facial emotion was mapped to one polarity, while the language model predicted a different polarity.

The project also includes an optional learned fusion model in `fusion.py`. This is a small PyTorch neural network that accepts a 10-dimensional input: seven image emotion probabilities and three text sentiment probabilities. It outputs a two-class decision: aligned or mismatched. This provides a pathway toward future trainable fusion while keeping the current default behaviour transparent.

6. Implementation Details

6.1 User Interface

The application is built in Streamlit. The UI is organised into six tabs:

- Overview
- Upload Image Analysis
- Live Webcam Analysis
- Analysis History
- Action Plan
- Future Roadmap

The overview tab visualises the core idea:

Happy face + text/audio = match or mismatch

The upload and webcam tabs share a common `text_audio_box()` component. This component handles typed text, recorded audio, uploaded audio, transcript display, and audio clearing. If the user provides audio and a transcript is successfully generated, typed text is no longer required. If no transcript exists, typed text remains mandatory.

6.2 Image Upload Workflow

The image upload workflow is:

- Users upload a .jpg, .jpeg, or .png file.
 - The app loads it with PIL and converts it to RGB.
 - OpenCV detects face regions.
 - The ViT model predicts emotion probabilities.
 - The annotated image and charts are displayed.
 - If enabled, an attention heatmap is generated.
 - The final result is fused with language sentiment.
-

6.3 Webcam Workflow

The webcam workflow is:

- The user starts the OpenCV camera.
- The app displays a live preview.
- OpenCV detects faces in each frame.
- A lightweight live mood overlay is shown.
- The user captures a clean frame.
- The captured frame is analysed like an uploaded image.

This design avoids requiring heavier real-time WebRTC dependencies in the main application workflow.

6.4 Text and Audio Workflow

The language workflow is:

- Users type text, records audio, or uploads audio.
- If audio is present, Whisper attempts transcription.
- The transcript is displayed in the UI.
- The app enables analysis when there is either typed text or transcript text.
- The full language input is sent to the RoBERTa sentiment model.

The latest version adds a Clear audio button. This is important because Streamlit widgets preserve state across reruns. The implementation clears audio by changing the widget keys through a reset counter, which causes Streamlit to render fresh audio widgets.

6.5 Explainability

MoodSync Studio includes two explainability tools:

Image heatmap: uses attention rollout from the ViT attention layers and overlays the strongest regions on the image.

Token attention: uses the final RoBERTa attention weights, averages attention heads, and displays token-level influence.

These do not prove model correctness, but they make the model outputs easier to inspect and discuss.

7. Pretrained Model Justification

7.1 Initial Training Attempt

During development, custom model training was explored using different emotion-related datasets and local training experiments. The objective was to train or fine-tune a custom model for facial emotion recognition and potentially improve control over the project pipeline. However, training modern deep-learning models became impractical on the available hardware.

The main problems were:

- Long training time on CPU
 - Insufficient GPU memory for larger image or transformer models
 - Slow experimentation when preprocessing and training repeatedly
 - Difficulty achieving stable performance without a large, balanced, cleaned dataset
 - Risk of overfitting on small or imbalanced emotion datasets
-

7.2 Why GPU Compute Matters

Modern models such as ViT, RoBERTa, and Whisper are computationally expensive because they contain millions of parameters and process high-dimensional data. The RoBERTa paper explicitly discusses how language model pretraining is computationally expensive and highly affected by data size and hyperparameters [3]. Whisper's official repository lists GPU memory requirements for different model sizes, with the base model requiring approximately 1 GB VRAM and larger models requiring much more [5]. Vision Transformer models also rely on large-scale pretraining before transfer to downstream image tasks [1].

Because of these constraints, using pretrained models was the most practical and technically responsible choice. It allowed the project to focus on:

- Multimodal integration

- User-facing workflow
 - Fusion logic
 - Explainability
 - Robust local demonstration
-

7.3 Benefits of Pretrained Models

Using pretrained models provided several benefits:

- Faster development
- Better baseline accuracy than a small custom-trained model
- Reduced training cost
- Ability to run on local CPU/GPU environments
- Access to established transformer and vision architectures
- More time to implement multimodal fusion and UI features

The project therefore follows a transfer-learning and model-integration approach instead of a full model-training approach.

8. Results and Testing

8.1 Functional Testing

Test Area	Expected Result
App syntax	Python files compile without syntax errors
Image upload	Uploaded image is displayed and prepared for analysis
Webcam capture	Camera can start, stop, and capture a clean frame
Text-only analysis	Analysis enabled when image/frame and text are present

Audio-only analysis	Analysis enabled when image/frame and transcript are present
Clear audio	Recorded/uploaded audio can be reset before typing text
Fusion	Result returns ALIGNED or MISMATCH
History	Completed analyses appear in session history
Action plan	Latest analysis can generate recommendations and report content

The implementation was checked using:

```
bash
python -m py_compile app.py image_emotion.py
```

8.2 Model Performance

This project does not claim newly trained model accuracy. Instead, it relies on reported performance and public model cards for the selected pretrained models.

The facial emotion model card reports a classification accuracy of approximately 0.9092 on its listed evaluation table and identifies the model as a ViT-based image classifier [2]. The CardiffNLP sentiment model card describes the model as RoBERTa-base trained on approximately 124 million tweets and fine-tuned for TweetEval sentiment analysis [4].

Formal end-to-end multimodal evaluation was not performed because this would require a labelled dataset containing aligned face, text, and audio examples. Future work should include a curated test set with human-labelled aligned and mismatched samples.

9. Ethical Considerations

Emotion recognition is sensitive because the output can influence how people are judged. Facial emotion models can be affected by demographic bias, lighting, camera quality, skin tone representation, pose, disability, cultural expression differences, and dataset limitations.

MoodSync Studio should not be used for:

- Medical diagnosis
- Hiring decisions
- Academic grading
- Policing or surveillance
- Detecting deception
- Making automatic decisions about a person

The application should be used only as an educational demonstration or decision-support prototype. Human interpretation is required, and users should be informed whenever their image, voice, or text is being analysed.

10. Limitations

The main limitations are:

- No custom end-to-end multimodal model was trained
- The rule-based fusion layer is explainable but simple
- The optional learned fusion model is lightweight and not validated on a real aligned multimodal dataset
- Audio is converted to text only; vocal tone and prosody are not analysed
- Face detection can fail under poor lighting, occlusion, or unusual camera angles
- Sentiment analysis can fail on sarcasm, irony, mixed emotions, and domain-specific language
- Confidence scores are not calibrated across modalities

- The system is not clinically validated
-

11. Future Enhancements

Future work can improve the project in the following ways:

- Train a real multimodal fusion model using labelled aligned and mismatched examples
 - Add audio prosody features such as pitch, loudness, pause length, speaking rate, and tone
 - Add calibration for emotion and sentiment confidence scores
 - Add bias testing with demographically balanced image samples
 - Add multilingual support using multilingual text and speech models
 - Add a privacy-first deployment mode with local-only processing and no data retention
 - Add automated unit tests and integration tests for every pipeline module
 - Add a formal evaluation report with accuracy, precision, recall, F1-score, and confusion matrices
 - Add continuous video timeline analysis for longer webcam sessions
 - Improve error handling for unsupported audio formats and weak microphone input
-

12. Conclusion

MoodSync Studio demonstrates how pretrained vision, language, and speech models can be combined into a practical multimodal AI application. The system analyses facial emotion and typed or spoken language, compares their polarity, and provides an interpretable aligned or mismatch result.

The project intentionally uses pretrained models because full model training was not feasible with the available computational resources. This decision allowed the work to

focus on the application layer, multimodal fusion, explainability, and user workflow. The resulting system is suitable as an educational prototype and a foundation for future work in multimodal affective computing.

References

- [1] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929. <https://arxiv.org/abs/2010.11929>
 - [2] dima806. facial_emotions_image_detection. Hugging Face Model Card. https://huggingface.co/dima806/facial_emotions_image_detection
 - [3] Liu, Y., Ott, M., Goyal, N., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692. <https://arxiv.org/abs/1907.11692>
 - [4] CardiffNLP. twitter-roberta-base-sentiment-latest. Hugging Face Model Card. <https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest>
 - [5] OpenAI. Whisper: Robust Speech Recognition via Large-Scale Weak Supervision. GitHub repository and paper links. <https://github.com/openai/whisper>
 - [6] Streamlit. st.audio_input API Reference. https://docs.streamlit.io/develop/api-reference/widgets/st.audio_input
 - [7] OpenCV. Cascade Classifier Tutorial. https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
 - [8] Baltrusaitis, T., Ahuja, C., and Morency, L. P. (2019). Multimodal Machine Learning: A Survey and Taxonomy. IEEE Transactions on Pattern Analysis and Machine Intelligence, 41(2), 423-443. <https://doi.org/10.1109/TPAMI.2018.2798607>
 - [9] Hugging Face. Transformers Pipelines Documentation. https://huggingface.co/docs/transformers/main_classes/pipelines
-

Appendix A: Dependency Summary

Library	Purpose
Streamlit	Web UI and interactive widgets
PyTorch	Model inference and optional fusion network
Transformers	Loading ViT and RoBERTa models
OpenAI Whisper	Speech-to-text transcription
OpenCV	Face detection and webcam handling
Pillow	Image loading and RGB conversion
NumPy	Array processing
Plotly	Probability and attention charts
SoundFile	Audio decoding
Librosa	Audio resampling

Appendix B: Reproducibility Notes

To reproduce the project:

```
bash
python -m venv .venv
```

Windows PowerShell:

```
powershell  
  
.\.venv\Scripts\Activate.ps1
```

Install dependencies:

```
bash  
  
pip install -r requirements.txt
```

Run the application:

```
bash  
  
python -m streamlit run app.py
```

The first run requires internet access to download pretrained model weights.

Github Link : <https://github.com/SidharthA42/Mood-Sync-Studio>

Deployed Link: <https://moodsyncstudio.streamlit.app/>